Total Variation Image Deblurring

An Extension of the Rudin–Osher–Fatemi Model via Convex Optimisation

Non-blind image reconstruction with synthetic degradation, isotropic TV regularisation, and a row-normalised sparse blur operator solved via CVXPY/SCS.

Reynaldi Aditama, Nam Bui, David Ou

March – April 2026 · MATH 441 — Convex Optimisation, UBC

Contents

1 Introduction **3**
- 1.1 Why Bother Recovering a Blurry Image? . . . 3
- 1.2 The Rudin–Osher–Fatemi Model . . . 3
- 1.3 Our Extension . . . 4

2 Project Setup **4**
- 2.1 Synthetic Degradation . . . 4
- 2.2 Image Choice and Preprocessing . . . 5
  - 2.2.1 Why the Cameraman Image? . . . 5
  - 2.2.2 Why Resize to 128×128? . . . 5
  - 2.2.3 Why Normalise to $[0, 1]$? . . . 5

3 Mathematical Model At a Glance **6**
- 3.1 Complete Formulation . . . 6
- 3.2 Decision Variable and Parameters . . . 6
- 3.3 Why This Problem Is Convex . . . 6

4 Mathematical Model In Depth **7**
- 4.1 The Data Fidelity Term . . . 7
  - 4.1.1 Where It Comes From . . . 7
  - 4.1.2 Why the Squared L^2 Norm, Not $\|\cdot\|_2$? . . . 7
  - 4.1.3 The Blur Matrix K and Why We Row-Normalise It . . . 7
- 4.2 The Total Variation Regularisation Term . . . 8
  - 4.2.1 What Total Variation Measures . . . 8
  - 4.2.2 Why TV Is Good at Preserving Edges . . . 8
  - 4.2.3 Isotropic vs. Anisotropic TV . . . 9
  - 4.2.4 The Role of λ . . . 9
- 4.3 Other Modelling Decisions . . . 9
  - 4.3.1 Why Not Estimate the Blur Kernel Too? . . . 9
  - 4.3.2 Why Keep the L^2 Data Term for Salt-and-Pepper Noise? . . . 10

5 Modelling Process **10**
- 5.1 The Degradation Pipeline in Code . . . 10
- 5.2 Building the Sparse Blur Matrix K . . . 10
- 5.3 CVXPY and the SCS Solver . . . 11

6 Results and Analysis **11**
- 6.1 Comparison Against Baselines . . . 11
- 6.2 Error Map Analysis . . . 12
- 6.3 Lambda Sensitivity Analysis . . . 12
- 6.4 Pareto Frontier: Data Fidelity vs. Total Variation . . . 13

7 Future Work and Extensions **14**
7.1 Colour Images 14
7.2 Scaling to Larger Images 14
7.3 A More Efficient Solver: The FTVd Algorithm 14

8 This Model in Context **15**
8.1 Why Industry Does Not Use This Model Directly 15
8.2 The Evolution of Image Restoration 15
8.3 Why This Model Still Matters 16

1 Introduction

1.1 Why Bother Recovering a Blurry Image?

Every image captured in the real world is a corrupted version of some ideal scene. A camera slightly out of focus, a patient moving during an MRI scan, a satellite collecting photons through atmospheric turbulence — in each case the observed image $\mathbf{f}$ results from a clean image $\mathbf{u}^0$ passing through two successive degradations: a blurring convolution K and the addition of noise $\boldsymbol{\omega}$. Recovering $\mathbf{u}^0$ from $\mathbf{f}$ is called *image deblurring*.

The naive approach — computing $K^{-1}\mathbf{f}$ — fails catastrophically. To see why, think about what blurring does: it averages each pixel with its neighbours, which smears sharp edges but also heavily suppresses fine details and high-frequency content. Inverting this process requires "un-averaging," which amplifies any small errors in $\mathbf{f}$ (such as noise) to the point where the result is visually meaningless. More formally, K is a severely ill-conditioned matrix, meaning its smallest singular values are near zero, and K^{-1} blows up the corresponding directions in the data.

The standard remedy is *regularisation*: instead of inverting K directly, we solve a minimisation problem that balances fitting the data $\mathbf{f}$ against a penalty that rewards images we consider "plausible." Choosing the right penalty is the central question in image restoration.

1.2 The Rudin–Osher–Fatemi Model

The foundational work this project builds on is the 1992 paper by Rudin, Osher, and Fatemi (ROF) [2], which proposed using *total variation* (TV) as the penalty. The idea is simple: natural images are mostly smooth, but they do have sharp edges (the boundary of an object, a horizon line). A good regulariser should allow those sharp edges to exist while discouraging random pixel-to-pixel noise.

TV does exactly this. It measures the total "amount of variation" in an image — how much the pixel values rise and fall in total across the image. A smooth region contributes almost nothing to TV; a sharp edge contributes proportionally to its contrast. Importantly, TV does *not* penalise a single clean edge more than a noisy region of the same total contrast, so it naturally preserves edges while smoothing out noise.

This is in contrast to the older Tikhonov regulariser, which penalises the squared L^2 norm of the gradient, $\|\nabla u\|_2^2$. Squaring the gradient means even a single large jump is penalised disproportionately, so the minimiser tends to smear every edge — exactly what we do not want.

The original ROF model for pure denoising (no blur, $K = I$) is:

$$\min_u \int_\Omega |\nabla u| \, dx \, dy \quad \text{s.t.} \quad \int_\Omega u = \int_\Omega u_0, \quad \int_\Omega (u - u_0)^2 = \sigma^2, \tag{1}$$

where u_0 is the noisy observation and σ is the noise standard deviation. The constraints enforce that the denoised image has the same mean as the observation and that the residual $u - u_0$ has variance matching the noise level.

A more convenient unconstrained version, and the form we use, folds the noise constraint into the objective as a penalty:

$$\min_{u} \ \frac{1}{2}\|Ku - \mathbf{f}\|_2^2 + \lambda \, \mathrm{TV}(u), \tag{2}$$

known as the TV/L^2 model [1]. The scalar $\lambda > 0$ replaces the Lagrange multiplier of the noise constraint and controls the trade-off between fitting the data and smoothing the image.

1.3 Our Extension

We extend the ROF denoising model in two directions:

(i) **We add a blur operator K.** The original ROF handles pure noise. We replace the identity with a Gaussian blur matrix, so the model must simultaneously undo blurring and remove noise. The blur kernel is *known* ("non-blind"), which keeps the problem convex.

(ii) **We use salt-and-pepper noise.** On top of blur we corrupt random pixels to pure black or white, simulating sensor imperfections. We keep the L^2 data term (which formally assumes Gaussian noise) because it keeps the full problem convex and well-suited to standard solvers.

The complete discrete formulation we solve is:

$$\min_{\mathbf{u}\in\mathbb{R}^{N^2}} \ \frac{1}{2}\|K\mathbf{u} - \mathbf{f}\|_2^2 + \lambda \sum_{i,j} \sqrt{(D_x\mathbf{u})_{ij}^2 + (D_y\mathbf{u})_{ij}^2} \quad \text{s.t.} \quad \mathbf{0} \le \mathbf{u} \le \mathbf{1}, \tag{3}$$

where $N = 128$, K is a sparse blur matrix, D_x and D_y are finite difference operators (pixel differences in the horizontal and vertical directions), and the box constraint enforces that pixel values stay in the valid range.

2 Project Setup

2.1 Synthetic Degradation

Because we work in a controlled setting, we generate the degraded image $\mathbf{f}$ ourselves rather than using a physical photograph. This is called *synthetic degradation*: we apply a known degradation to a clean image, then try to recover the original. The key advantage is that we have access to the ground truth $\mathbf{u}^0$, which lets us measure reconstruction quality exactly.

Our pipeline applies two steps:

Step 1 — Gaussian blur ($\sigma = 3.0$, kernel size 11×11). We convolve $\mathbf{u}^0$ with a Gaussian kernel $k(x, y) \propto \exp\big(-(x^2 + y^2)/(2\sigma^2)\big)$, normalised so all weights sum to 1. Think of this as replacing each pixel by a weighted average of its 11×11 neighbourhood, with more weight given to closer pixels. At $\sigma = 3.0$, the blur radius is about three pixels, which is enough to visibly smear fine structure.

Step 2 — Salt-and-pepper noise ($p = 0.05$). Each pixel is independently set to 0 ("pepper", pure black) with probability $p/2$ and to 1 ("salt", pure white) with probability $p/2$, leaving the remaining 95% of pixels untouched. With $p = 0.05$, roughly 1 in 20 pixels is corrupted.

After both steps the image is clipped to $[0, 1]$. The resulting $\mathbf{f}$ achieves PSNR = 16.60 dB and SSIM = 0.2290 relative to the clean image — both metrics are explained in Section 6.

2.2 Image Choice and Preprocessing

2.2.1 Why the Cameraman Image?

We use the standard cameraman photograph, a 512×512 grayscale image widely used as a benchmark in image processing. It contains smooth regions (sky, uniform clothing), high-contrast edges (camera outline, tripod), and fine texture (distant trees). This variety is useful: a method that works only on flat regions or only on edges will fail on part of the image, which shows up clearly in the metrics.

2.2.2 Why Resize to 128×128?

The main reason is computational. Our optimisation variable is $\mathbf{u} \in \mathbb{R}^{N^2}$, and the blur matrix K has size $N^2 \times N^2$. At $N = 128$, that is a $16{,}384 \times 16{,}384$ matrix — already large enough that the solver takes about 40 seconds. Doubling to $N = 256$ would give a $65{,}536 \times 65{,}536$ matrix and roughly a $16\times$ increase in solve time. The λ-sensitivity and Pareto experiments use $N = 64$ for the same reason: each value of λ needs a fresh solve.

2.2.3 Why Normalise to $[0, 1]$?

The image is normalised via

$$\mathbf{u}^0 \leftarrow \frac{\mathbf{u}^0 - \min(\mathbf{u}^0)}{\max(\mathbf{u}^0) - \min(\mathbf{u}^0)},$$

so pixel values lie in $[0, 1]$. This matters for two reasons. First, the box constraint $\mathbf{0} \leq \mathbf{u} \leq \mathbf{1}$ only makes physical sense when the data is in the same range. Second, the choice of λ has a consistent meaning: a penalty of $\lambda = 0.05$ means the same thing regardless of whether pixel values are on a $[0, 1]$ or a $[0, 255]$ scale if we always normalise first.

3 Mathematical Model At a Glance

3.1 Complete Formulation

$$\boxed{\min_{\mathbf{u}\in\mathbb{R}^{N^2}} \underbrace{\frac{1}{2}\|K\mathbf{u}-\mathbf{f}\|_2^2}_{\text{data fidelity}} + \lambda \underbrace{\sum_{i,j}\sqrt{(D_x\mathbf{u})_{ij}^2+(D_y\mathbf{u})_{ij}^2}}_{\text{isotropic total variation}} \quad \text{s.t.} \quad \mathbf{0}\le\mathbf{u}\le\mathbf{1}} \tag{4}$$

The objective has two competing parts. The data fidelity term asks that applying the blur to our recovered image $\mathbf{u}$ produces something close to what we observed, $\mathbf{f}$. The TV term asks that the recovered image is not excessively variable. The scalar λ sets how strongly we enforce the second requirement relative to the first.

3.2 Decision Variable and Parameters

$\mathbf{u}\in\mathbb{R}^{N^2}$ is the vectorised restored image: the pixel at row i, column j of the $N\times N$ grid corresponds to component $iN+j$ of $\mathbf{u}$.

Symbol	Value / size	Meaning
N	128	image side length
K	$\mathbb{R}^{N^2\times N^2}$, sparse	row-normalised Gaussian blur matrix
$\mathbf{f}$	$\mathbb{R}^{N^2}$	observed (blurred + noisy) image, vectorised
λ	0.05 (main experiment)	regularisation strength
σ	3.0	Gaussian blur standard deviation (pixels)
p	0.05	salt-and-pepper noise probability

3.3 Why This Problem Is Convex

The objective (4) is a sum of two terms:

- $\frac{1}{2}\|K\mathbf{u}-\mathbf{f}\|_2^2$ is a convex quadratic in $\mathbf{u}$ (it is a squared norm composed with a linear function).
- $\lambda\,\mathrm{TV}(\mathbf{u})$ is convex because each summand $\sqrt{a^2+b^2}$ is a norm of the vector (a,b), and norms are always convex.

The feasible set $[\mathbf{0},\mathbf{1}]$ is a box, which is convex. A sum of convex functions over a convex feasible set is a convex programme, meaning every local minimum is a global minimum. In practice, CVXPY confirms this automatically (output: `Problem is DCP: True`).

4 Mathematical Model In Depth

4.1 The Data Fidelity Term

4.1.1 Where It Comes From

The data fidelity term $\frac{1}{2}\|K\mathbf{u} - \mathbf{f}\|_2^2$ has a clear interpretation: it measures how much the "predicted observation" $K\mathbf{u}$ differs from what we actually observed, $\mathbf{f}$.

This term is derived from the forward model. We assume the observed image was generated as

$$\mathbf{f} = K\mathbf{u}^0 + \boldsymbol{\omega},$$

where $\mathbf{u}^0$ is the clean image we want to recover and $\boldsymbol{\omega}$ is noise. If the noise were Gaussian, $\boldsymbol{\omega} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$, then the probability of observing $\mathbf{f}$ given a candidate image $\mathbf{u}$ is

$$p(\mathbf{f} \mid \mathbf{u}) \propto \exp\left(-\frac{1}{2\sigma^2}\|K\mathbf{u} - \mathbf{f}\|_2^2\right).$$

Maximising this over $\mathbf{u}$ is the same as minimising $\|K\mathbf{u} - \mathbf{f}\|_2^2$, which is exactly our data fidelity term.

Our noise is actually salt-and-pepper, not Gaussian. We still use the L^2 term for a practical reason: it keeps the fidelity term *smooth and quadratic*, which makes the full problem much easier to solve. The trade-off is that the model is slightly misspecified for impulsive noise — and this shows up as small residual errors at corrupted pixels.

4.1.2 Why the Squared L^2 Norm, Not $\|\cdot\|_2$?

Using $\|\cdot\|_2^2$ (sum of squares) rather than $\|\cdot\|_2$ (square root of sum of squares) is a deliberate choice:

1. **Smooth gradient.** The gradient of $\frac{1}{2}\|K\mathbf{u} - \mathbf{f}\|_2^2$ with respect to $\mathbf{u}$ is simply $K^T(K\mathbf{u} - \mathbf{f})$. By contrast, $\|\cdot\|_2$ is not differentiable at zero, complicating analysis.

2. **The $\frac{1}{2}$ factor.** The pre-factor $\frac{1}{2}$ is a convenience: without it the gradient would have an extra factor of 2, cluttering every formula. It does not change the minimiser.

3. **Quadratic structure.** The $\mathbf{u}$-subproblem in iterative solvers reduces to a linear system when the data term is quadratic, which can be solved efficiently.

4.1.3 The Blur Matrix K and Why We Row-Normalise It

The blur matrix $K \in \mathbb{R}^{N^2 \times N^2}$ encodes the Gaussian convolution as a matrix multiplication. For an interior pixel (i, j), the corresponding row of K contains the kernel weights for all source pixels in the neighbourhood:

$$(K\mathbf{u})_{ij} = \sum_{p,q} k_\sigma(p, q)\, u_{i+p-r,\, j+q-r}, \quad r = \lfloor \text{kernel size}/2 \rfloor.$$

This is just the discrete convolution formula written out explicitly.

The complication arises at *boundary pixels*: part of the 11×11 kernel support falls outside the image. If we simply ignore out-of-bounds pixels (zero-padding), the kernel weights that do lie inside the image sum to less than 1. This means a pixel near the edge sees a darker-than-expected average, which introduces a systematic bias at boundaries. To fix this, we *row-normalise*: after building K, each row is divided by its sum,

$$K_{\text{norm}}[i, \cdot] \leftarrow \frac{K[i, \cdot]}{\sum_j K[i, j]},$$

so every row sums to exactly 1 regardless of boundary position. After normalisation, the matrix has shape $16{,}384 \times 16{,}384$ with about 1.9 million nonzeros (only $\approx 0.71\%$ of entries are nonzero, so we store it in sparse format).

4.2 The Total Variation Regularisation Term

4.2.1 What Total Variation Measures

For a continuous image u defined on a domain Ω, the total variation is

$$\text{TV}(u) = \int_\Omega |\nabla u| \, dx \, dy = \int_\Omega \sqrt{u_x^2 + u_y^2} \, dx \, dy.$$

Intuitively, $|\nabla u|$ at a point measures how fast the pixel value is changing there. TV is the total amount of change summed over the whole image.

For a discrete $N \times N$ image $\mathbf{u}$, we approximate the gradient at pixel (i, j) using first-order finite differences:

$$(D_x \mathbf{u})_{ij} = u_{i,\,j+1} - u_{i,j}, \qquad (D_y \mathbf{u})_{ij} = u_{i+1,\,j} - u_{i,j}.$$

These are just the pixel difference to the right and below. The isotropic discrete TV is then:

$$\text{TV}_{\text{iso}}(\mathbf{u}) = \sum_{i=1}^{N} \sum_{j=1}^{N} \sqrt{(D_x \mathbf{u})_{ij}^2 + (D_y \mathbf{u})_{ij}^2}. \tag{5}$$

Each term under the sum is the magnitude of the gradient vector at pixel (i, j). Summing these magnitudes gives the total amount of variation across the entire image.

4.2.2 Why TV Is Good at Preserving Edges

The key property of TV regularisation is that it penalises variation *linearly* in gradient magnitude. Compare this to Tikhonov regularisation, which penalises $\sum_{i,j} \left[(D_x \mathbf{u})_{ij}^2 + (D_y \mathbf{u})_{ij}^2 \right]$ — the squared gradient. Squaring amplifies large gradients disproportionately, so the minimiser under Tikhonov regularisation strongly prefers to reduce large jumps, which means it blurs edges. Under TV, a gradient of magnitude 10 costs 10 times as much as one of magnitude 1, but not 100 times as much. The solver is therefore much more willing

to allow a single large jump (a sharp edge) if smoothing it out would require spreading variation across many pixels.

Formally, this is related to the fact that the TV functional is defined on the space $\mathrm{BV}(\Omega)$ of *functions of bounded total variation*, which explicitly allows discontinuities. Tikhonov regularisation, being quadratic, lives in the Sobolev space $H^1(\Omega)$, which does not allow jumps. In plain terms: you simply cannot write down a function with a sharp edge that has finite Tikhonov energy, but you can write one with finite TV.

4.2.3 Isotropic vs. Anisotropic TV

There are two common discrete forms of TV. *Anisotropic* TV uses the ℓ^1 norm of the gradient vector:

$$\mathrm{TV}_{\mathrm{aniso}}(\mathbf{u}) = \sum_{i,j} \big(|(D_x \mathbf{u})_{ij}| + |(D_y \mathbf{u})_{ij}| \big).$$

Isotropic TV (which we use) uses the ℓ^2 norm, as in (5).

The practical difference comes down to how diagonal edges are treated. Under anisotropic TV, a diagonal edge of gradient magnitude g contributes $|g\cos\theta| + |g\sin\theta|$ to the total, which depends on the angle θ of the edge. A perfectly diagonal edge ($\theta = 45$) costs $g/\sqrt{2} + g/\sqrt{2} = g\sqrt{2}$, more than a horizontal edge of the same magnitude. The minimiser therefore tends to represent diagonal transitions as staircase patterns of horizontal and vertical steps — a visible artifact.

Under isotropic TV, the contribution is $\sqrt{(g\cos\theta)^2 + (g\sin\theta)^2} = g$ regardless of angle. The penalty is the same for edges in any direction, which avoids the staircase artifact. This rotation-invariance is why isotropic TV is the standard choice for photographic images [1]. CVXPY implements this via `cp.tv(u_2d)`.

4.2.4 The Role of λ

The parameter $\lambda > 0$ balances the two objectives:

- **Small** λ: the data term dominates. The solver fits $\mathbf{f}$ very closely, reproducing whatever noise is present in the observation.
- **Large** λ: the TV term dominates. The solver aggressively smooths the image, losing fine detail to reduce variation.

There is no universally correct λ — it depends on the noise level and the image content. In our experiments (Section 6.3), we sweep λ from 0.005 to 0.5 and find the best PSNR at $\lambda = 0.02$.

4.3 Other Modelling Decisions

4.3.1 Why Not Estimate the Blur Kernel Too?

When both the image and the blur kernel are unknown, the problem is called *blind deconvolution*. It is considerably harder: the product $K\mathbf{u}$ is bilinear (linear in each variable

separately, but not jointly), so the joint problem over $(K, \mathbf{u})$ is *non-convex*. Non-convex problems can have many local minima, and standard solvers can get stuck in a bad one.

By fixing K to the known Gaussian kernel, the problem becomes convex in $\mathbf{u}$ alone. This means any local minimum is the global minimum — a much stronger guarantee. Our goal is to study TV regularisation as a mathematical framework, and fixing K cleanly isolates that effect.

4.3.2 Why Keep the L^2 Data Term for Salt-and-Pepper Noise?

Salt-and-pepper noise corrupts pixels to pure 0 or 1 — these are large outliers, not the small symmetric errors that the Gaussian model assumes. An L^1 data term (which penalises $\|K\mathbf{u}-\mathbf{f}\|_1$ rather than the square) would be more appropriate statistically, because it is less sensitive to extreme values. However, an L^1 data term makes the problem harder to solve: it is not differentiable, and solvers for it are slower and less mature than those for L^2. Since our primary goal is to study the regularisation term rather than the fidelity term, we accept the slight model mismatch and use L^2 throughout.

5 Modelling Process

5.1 The Degradation Pipeline in Code

The notebook applies the following steps:

1. Load the cameraman image, resize to 128×128 with anti-aliased bilinear interpolation, and normalise to $[0, 1]$.
2. Generate the Gaussian kernel: place a δ-impulse at the centre of an 11×11 grid, filter it with `scipy.ndimage.gaussian_filter` at $\sigma = 3$, and normalise to sum to 1.
3. Apply the blur via `gaussian_filter(u_true, sigma=3)`.
4. Add salt-and-pepper noise: draw a uniform random matrix; set pixels to 0 where the draw is below $p/2$ and to 1 where it exceeds $1 - p/2$.
5. Clip to $[0, 1]$ to handle any numerical overshoot.

5.2 Building the Sparse Blur Matrix K

Gaussian blur is most efficiently applied as a 2D convolution using `gaussian_filter`. However, CVXPY needs the problem written as a matrix expression, so we must build K explicitly. We construct it as a sparse LIL matrix (list-of-lists format) and then convert to CSR format for efficient multiplication.

For each output pixel (i, j) and each kernel position (k_i, k_j), the entry

$$K[\,iN + j,\ (i + k_i - r)N + (j + k_j - r)\,] \mathrel{+}= k_\sigma(k_i, k_j)$$

is set whenever the source pixel lies inside the image. Each row is then divided by its sum (row normalisation). Construction takes about 3 seconds; the final matrix has 1,898,884 nonzeros at 0.71% density.

5.3 CVXPY and the SCS Solver

CVXPY (v1.8.2) is a Python library for specifying and solving convex optimisation problems. The user writes the problem in mathematical notation and CVXPY automatically transforms it into a form that a numerical solver can handle.

We use the SCS (Splitting Conic Solver) backend, which solves the problem using the alternating direction method of multipliers (ADMM). SCS is well-suited to large-scale problems that combine smooth and non-smooth terms. The problem definition in code is:

```
u     = cp.Variable(N * N)
u_2d  = cp.reshape(u, (N, N), order='C')
obj   = 0.5 * cp.sum_squares(K @ u - f_flat)
      + LAMBDA * cp.tv(u_2d)
prob  = cp.Problem(cp.Minimize(obj), [u >= 0, u <= 1])
prob.solve(solver=cp.SCS)
```

With $N = 128$, this has 16,384 decision variables. The solver reports status `optimal` with objective value 134.28, taking approximately 40 seconds.

6 Results and Analysis

We use two standard metrics to measure how close a recovered image $\mathbf{u}$ is to the ground truth $\mathbf{u}^0$:

- **PSNR (Peak Signal-to-Noise Ratio):** $\text{PSNR} = 10\log_{10}\big(1/\text{MSE}\big)$, where $\text{MSE} = \frac{1}{N^2}\|\mathbf{u}-\mathbf{u}^0\|_2^2$. It is measured in decibels; higher is better. Values below 20 dB indicate severe quality loss; above 30 dB is generally considered good for grayscale images.
- **SSIM (Structural Similarity Index):** A perceptual metric in $[0, 1]$ that compares local luminance, contrast, and structural patterns rather than pixel-level differences. SSIM closer to 1 means the image looks more similar to the original. Unlike PSNR, it is sensitive to whether edges are in the right places, not just whether pixel values are close.

6.1 Comparison Against Baselines

We compare our TV model against three methods: the degraded input itself (no recovery), a Gaussian filter ($\sigma = 1.5$) applied directly to $\mathbf{f}$, and a 3×3 median filter applied to $\mathbf{f}$.

Table 1: Reconstruction quality on the 128×128 cameraman image. Bold marks the best value in each column.

Method	PSNR (dB)	SSIM
Degraded (blurred + S&P noise)	16.60	0.2290
Gaussian filter ($\sigma = 1.5$)	21.06	0.5278
Median filter (3×3)	22.04	0.6674
TV model (ours) $\lambda = 0.05$	**22.53**	0.6590

The TV model achieves the highest PSNR (22.53 dB), a 5.93 dB gain over the degraded input. In SSIM, it comes second to the median filter (0.6590 vs. 0.6674).

The comparison with the median filter is instructive. Median filtering replaces each pixel with the median of its 3×3 neighbourhood, which is specifically designed for salt-and-pepper noise: since only 5% of pixels are corrupted, most 3×3 windows contain mostly uncorrupted pixels, so the median is a clean value. The TV model, using an L^2 data term, treats the corrupted pixels as large Gaussian outliers, which it cannot cancel as cleanly. However, the TV model beats the median filter on PSNR because it also tackles the blur — the median filter does nothing to undo the Gaussian convolution, while the TV model jointly handles both.

6.2 Error Map Analysis

The absolute error map $|\mathbf{u} - \mathbf{u}^0|$ at each pixel reveals *where* a method fails, not just how much it fails overall.

- The **degraded image** shows two kinds of error: large isolated spikes at the salt-and-pepper corrupted pixels, and diffuse low-level error throughout smooth regions from the blur.
- The **Gaussian filter** reduces the diffuse blur error (it already does something blur-like) but introduces its own smooth haze and cannot remove the S&P spikes well.
- The **median filter** efficiently removes the isolated S&P spikes, but the underlying blur-induced error remains essentially untouched.
- The **TV model** shows a different pattern: smooth regions are reconstructed very accurately (low error), but edges and textured areas (the tree line, camera body) retain more error. This is TV's signature behaviour — it excels at recovering piecewise-smooth content but cannot recover fine texture, which it interprets as variation to be penalised.

6.3 Lambda Sensitivity Analysis

To understand how λ affects the result, we run ten separate solves on the 64×64 version of the image with λ ranging from 0.005 to 0.5.

Table 2: TV model metrics on the 64×64 image across λ values. The fidelity column is $\frac{1}{2}\|K\mathbf{u} - \mathbf{f}\|_2^2$ evaluated at the solution.

λ	Fidelity	TV	PSNR (dB)	SSIM
0.005	29.08	160.39	19.39	0.4565
0.010	29.36	120.50	20.82	0.5552
0.020	29.67	98.34	**21.28**	0.6216
0.050	30.19	81.96	21.03	**0.6266**
0.080	30.67	74.43	20.66	0.6113
0.100	30.96	71.22	20.43	0.6025
0.150	31.50	66.80	20.04	0.5856
0.200	31.95	64.21	19.79	0.5790
0.300	32.92	60.27	19.32	0.5675
0.500	35.21	54.48	18.51	0.5448

The table tells a consistent story. As λ increases, the fidelity cost rises (the solution drifts further from $\mathbf{f}$) while TV decreases (the image becomes smoother). Both trends are monotone, as expected. The best PSNR is achieved at $\lambda = 0.02$ and the best SSIM at $\lambda = 0.05$ — a small but notable gap.

This gap reflects what the two metrics value. PSNR rewards pixel-level accuracy, so it prefers a smaller λ that stays closer to the data. SSIM rewards structural similarity — whether edges are in the right place and contrast is preserved — so it tolerates slightly more smoothing in exchange for a cleaner image structure.

At the extreme values, the model fails in different directions. At $\lambda = 0.005$ the solver barely regularises: the image fits $\mathbf{f}$ very closely, including all its noise. At $\lambda = 0.5$ the image is over-smoothed to the point of looking like a patchwork of flat regions, losing all texture detail.

6.4 Pareto Frontier: Data Fidelity vs. Total Variation

Each choice of λ produces a different solution $\mathbf{u}^*(\lambda)$, and therefore a different pair of objective values $\big(\frac{1}{2}\|K\mathbf{u}^* - \mathbf{f}\|_2^2,\ \mathrm{TV}(\mathbf{u}^*)\big)$. Plotting these pairs as λ varies traces the *Pareto frontier*:

$$\mathcal{P} = \Big\{\big(\tfrac{1}{2}\|K\mathbf{u}^* - \mathbf{f}\|_2^2,\ \mathrm{TV}(\mathbf{u}^*)\big) : \mathbf{u}^* \text{ solves (4) for some } \lambda > 0\Big\}.$$

A point on $\mathcal{P}$ is called Pareto-optimal if there is no other feasible $\mathbf{u}$ that achieves a lower fidelity cost *and* a lower TV simultaneously. In other words, to reduce one objective you must accept an increase in the other. The frontier separates achievable trade-offs from ideal ones that are impossible.

Concretely, from Table 2, moving from $\lambda = 0.005$ to $\lambda = 0.5$ moves the operating point from $(29.08, 160.39)$ to $(35.21, 54.48)$ in the (fidelity, TV) plane: fidelity gets worse (the solution fits $\mathbf{f}$ less closely) while TV improves (the image is smoother). There is no

point in the interior of the feasible region that simultaneously achieves lower fidelity cost and lower TV than both of these.

The PSNR-coloured Pareto plot reveals that the best reconstruction quality ($\lambda = 0.02$) lies in the middle of the frontier, not at either extreme. This is the characteristic behaviour of regularised inverse problems: there is a "sweet spot" where the data and the prior are in balance, and moving too far in either direction hurts.

7 Future Work and Extensions

7.1 Colour Images

The current model operates on a single-channel (grayscale) image. Extending to a three-channel colour image $\mathbf{u} \in \mathbb{R}^{N^2 \times 3}$ requires a regulariser that couples the three channels. The natural choice is *vectorial total variation*:

$$\mathrm{VTV}(\mathbf{u}) = \sum_{i,j} \sqrt{\sum_{c=1}^{3} (D_x \mathbf{u}^c)_{ij}^2 + (D_y \mathbf{u}^c)_{ij}^2},$$

which penalises the total gradient magnitude across all channels jointly at each pixel. The key property is that an edge which appears in all three channels (e.g. the sharp boundary of a red object on a white background) is penalised once, not three times, preventing colour fringing at edges [1].

7.2 Scaling to Larger Images

Our implementation is limited to 128×128 by the cost of storing and using K explicitly. A 256×256 image would produce a $65{,}536 \times 65{,}536$ matrix. Even in sparse format, passing it to CVXPY/SCS would take far longer.

The FTVd algorithm of Wang et al. [1] avoids this by exploiting the fact that Gaussian blur is a *convolution*: in the Fourier domain, convolution becomes pointwise multiplication, and the $\mathbf{u}$-update that requires solving a linear system can instead be computed with three FFTs. This reduces the per-iteration cost from $O(N^4)$ (dense matrix solve) to $O(N^2 \log N)$ (FFT), making 1024×1024 images tractable.

7.3 A More Efficient Solver: The FTVd Algorithm

The FTVd algorithm [1] solves our TV/L^2 model by introducing an auxiliary variable $\mathbf{w}_i \in \mathbb{R}^2$ to approximate the image gradient $D_i \mathbf{u}$ at each pixel i:

$$\min_{\mathbf{w},\mathbf{u}} \sum_i \|\mathbf{w}_i\|_2 + \frac{\beta}{2} \sum_i \|\mathbf{w}_i - D_i \mathbf{u}\|_2^2 + \frac{\mu}{2} \|K\mathbf{u} - \mathbf{f}\|_2^2.$$

The coupling $\|\mathbf{w}_i - D_i \mathbf{u}\|^2$ is a penalty that forces $\mathbf{w}$ to track the gradient of $\mathbf{u}$; as the penalty parameter $\beta \to \infty$, the solution converges to the original TV model [1].

The advantage is that this problem can be minimised by alternating between two simple steps:

- A **w-step**: for each pixel, compute the 2D soft-threshold (shrinkage) of the gradient, which has the closed form $\mathbf{w}_i^* = \max\big(\|D_i\mathbf{u}\|_2 - 1/\beta,\, 0\big) \cdot D_i\mathbf{u}/\|D_i\mathbf{u}\|_2$. This step processes each pixel independently in $O(N^2)$ total.

- A **u-step**: solve a linear system whose coefficient matrix has block-circulant structure (because both K and D are convolutions), which diagonalises under the 2D Fourier transform. The solution requires three FFTs.

The total solve uses roughly 40 FFTs and runs in under one second for 512×512 images — versus 40 seconds for our CVXPY/SCS approach on 128×128 [1].

8 This Model in Context

8.1 Why Industry Does Not Use This Model Directly

The TV/L^2 model is mathematically clean but has practical limitations in commercial image processing:

1. **Speed.** Even with FTVd, solving an optimisation problem per image is orders of magnitude slower than a single forward pass through a neural network.

2. **Staircase artifacts.** TV promotes piecewise-constant solutions. Smooth gradients (sky, skin) are represented as flat regions with abrupt jumps, producing a characteristic "painted" or "cartoon" appearance in smooth areas.

3. **Parameter tuning.** The right λ is image-dependent. A fixed value that works well for one image may over- or under-smooth another.

4. **Known blur.** We assumed the blur kernel K is known exactly. Real cameras have complex, spatially-varying PSFs that are hard to measure.

8.2 The Evolution of Image Restoration

TV regularisation can be seen as the first principled step in a longer progression.

The next step was replacing the hand-designed TV prior with a *learned* one. Plug-and-play priors (PnP-ADMM) embed a trained image denoiser into an iterative ADMM scheme, replacing the TV proximal step with a call to the denoiser. This keeps the iterative structure (and thus interpretability) while gaining the expressiveness of a data-driven prior.

Eventually, end-to-end convolutional neural networks (CNNs) took over for most practical applications: DnCNN (2017) and similar models learn the entire restoration mapping from blurry/noisy to clean, implicitly learning a prior much richer than TV.

Modern state-of-the-art methods use *score-based diffusion models*, which learn the gradient of the log-probability of natural images from a large dataset. These produce perceptually stunning results but require 100–1000 neural network evaluations per image, making them slow relative to CNNs.

8.3 Why This Model Still Matters

Despite being superseded for consumer photography, the TV/L^2 model is still actively used in several domains:

1. **Medical imaging (MRI).** Compressed sensing MRI uses TV regularisation to reconstruct images from heavily undersampled measurements. Mathematical guarantees and interpretability are essential in clinical settings.

2. **Astronomy and microscopy.** The blur kernel is often known precisely (from a calibration star or a known point source), so non-blind TV deconvolution is a natural and well-principled choice.

3. **Engineering inverse problems.** CT reconstruction, electrical impedance tomography, and seismic imaging all share the same mathematical structure and benefit from TV priors.

4. **Compressed sensing.** Replacing the Gaussian blur operator K with a partial Fourier transform (sampling only some Fourier coefficients) gives the compressed sensing problem for images with sparse gradients. The FTVd algorithm [1] applies with minor modifications, showing how widely the framework generalises.

5. **Theory.** Convex TV models are the testing ground for proximal algorithms, split-Bregman iterations, and primal–dual methods. Understanding them is a foundation for the convergence analysis of more complex modern methods.

References

[1] Y. Wang, J. Yang, W. Yin, and Y. Zhang, "A New Alternating Minimization Algorithm for Total Variation Image Reconstruction," *SIAM Journal on Imaging Sciences*, vol. 1, no. 3, pp. 248–272, 2008.

[2] L. I. Rudin, S. Osher, and E. Fatemi, "Nonlinear total variation based noise removal algorithms," *Physica D: Nonlinear Phenomena*, vol. 60, no. 1–4, pp. 259–268, 1992.